

StyleGAN-V: A Continuous Video Generator with the Price, Image Quality and Perks of StyleGAN2

Ivan Skorokhodov
KAUST

Sergey Tulyakov
Snap Inc.

Mohamed Elhoseiny
KAUST

Abstract

Videos show continuous events, yet most — if not all — video synthesis frameworks treat them discretely in time. In this work, we think of videos of what they should be — time-continuous signals, and extend the paradigm of neural representations to build a continuous-time video generator. For this, we first design continuous motion representations through the lens of positional embeddings. Then, we explore the question of training on very sparse videos and demonstrate that a good generator can be learned by using as few as 2 frames per clip. After that, we rethink the traditional image and video discriminators pair and propose to use a single hypernetwork-based one. This decreases the training cost and provides richer learning signal to the generator, making it possible to train directly on 1024^2 videos for the first time. We build our model on top of StyleGAN2 and it is just $\approx 5\%$ more expensive to train at the same resolution while achieving almost the same image quality. Moreover, our latent space features similar properties, enabling spatial manipulations that our method can propagate in time. We can generate arbitrarily long videos at arbitrary high frame rate, while prior work struggles to generate even 64 frames at a fixed rate. Our model achieves state-of-the-art results on four modern 256^2 video synthesis benchmarks and one 1024^2 resolution one.¹

1. Introduction

Recent advances in deep learning pushed image generation to the unprecedented photo-realistic quality [8, 29] and spawned a lot of its industry applications. Video generation, however, does not enjoy a similar success and struggles to fit complex real-world datasets. The difficulties are caused not only by the more complex nature of the underlying data distribution, but also due to the computationally intensive video representations employed by modern generators. They treat videos as discrete sequences of images, which

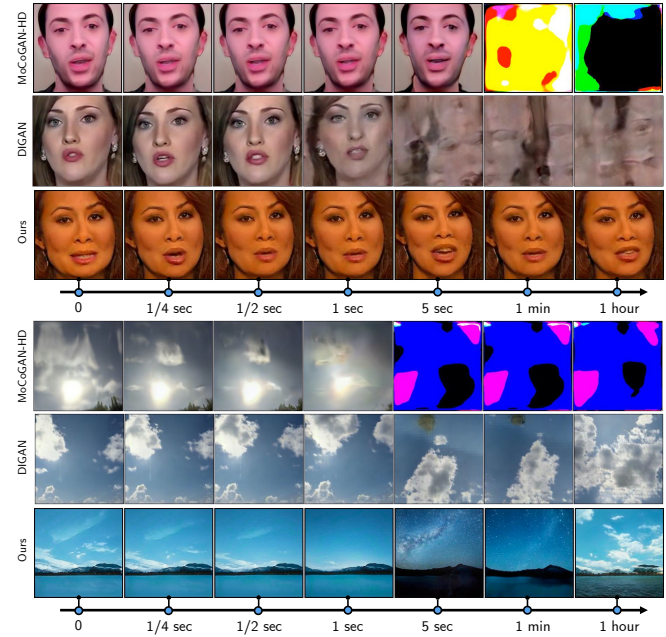


Figure 1. Examples of 1-hour long videos, generated with different methods. MoCoGAN-HD [67] fails to generate long videos due to the instability of the underlying LSTM model when unrolled to large lengths. DIGAN [5] struggles to generate long videos due to the entanglement of spatial and temporal positional embeddings. StyleGAN-V (our method) generates plausible videos of arbitrary length and frame-rate. Also, unlike DIGAN, it learns temporal patterns not only in terms of motion, but also appearance transformations, like time of day and weather changes.

is very demanding for representing long high-resolution videos and induces the use of expensive `conv3d`-based architectures to model them [14, 56, 57, 69].²

In this work, we argue that this design choice is not optimal and propose to treat videos in their natural form: as continuous signals $x(t)$, that map *any* time coordinate $t \in \mathbb{R}_+$ into an image frame $x(t) = x_t \in \mathbb{R}^{3 \times h \times w}$. Conse-

¹Videos and source code: <https://universome.github.io/stylegan-v>.

²E.g., DVD-GAN [14] requires $\approx \$30K$ to train at 256^2 resolution (as reported by [67]).

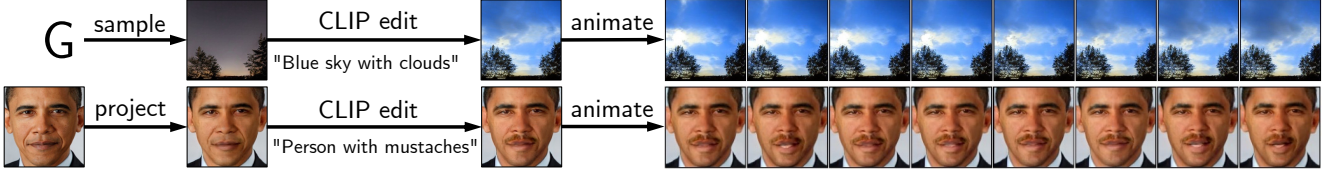


Figure 2. Our model enjoys all the perks of StyleGAN2 [31], including the ability of semantic manipulation. In this example, we edited a generated frame (top row) or projected off-the-shelf image (bottom row) with CLIP and animated it with our model. To the best of our knowledge, our work is the first one which demonstrates such capabilities for video generators.

quently, we develop a GAN-based continuous video synthesis framework by extending the recent paradigm of neural representations [40, 60, 66] to the video generation domain.

Developing such a framework comes with three challenges. First, sine/cosine positional embeddings are periodic by design and depend only on the input coordinates. This does not suit video generation, where temporal information should be aperiodic (otherwise, videos will be cycled) and different for different samples. Next, since videos are perceived as infinite continuous signals, one needs to develop an appropriate sampling scheme to use them in a practical framework. Finally, one needs to accordingly redesign the discriminator to operate in the new sampling pipeline.

To solve the first issue, we develop positional embeddings with time-varying wave parameters which depend on motion information, sampled uniquely for different videos. This motion information is represented as a sequence of motion codes produced by a *padding-less* conv1d-based model. We prefer it over the usual LSTM network [3, 57, 67, 69] to alleviate the RNN’s instability when unrolled to large depths and to produce frames non-autoregressively.

Next, we investigate the question of how many samples are needed to learn a meaningful video generator. We argue that it can be learned from *extremely* sparse videos (as few as 2 frames per clip), and justify it with a simple theoretical exposition in Sec 3.3 and practical experiments (see Table 2).

Finally, since our model sees only 2-4 randomly sampled frames per video, it is highly redundant to use expensive conv3d-blocks in the discriminator, which are designed to operate on long sequences of equidistant frames. That’s why we replace it with a conv2d-based model, which aggregates information temporarily via simple concatenation and is conditioned on the time distances between its input frames. We use hypernetwork-based modulation [21] for this conditioning to make the discriminator more flexible in processing frames sampled at varying time distances. Such redesign improves training efficiency (see Table 1), provides more informative gradient signal to the generator (see Fig 4) and simplifies the overall pipeline (see Sec 3.2), since we no longer need two different discriminators to operate on image and video levels separately, as modern video synthesis models do (e.g., [14, 57, 69]).

We build our model, named StyleGAN-V, on top of the image-based StyleGAN2 [31]. It is able to produce arbitrarily long videos at arbitrarily high frame-rate in a non-autoregressive manner and enjoys great training efficiency — it is only $\approx 5\%$ costlier than the classical *image-based* StyleGAN2 model [31], while having only $\approx 10\%$ worse *plain* image quality in terms of FID [23] (see Fig 3). This allows us to easily scale it to HQ datasets and we demonstrate that it is *directly* trainable on 1024^2 resolution.

For empirical evaluation, we use 5 benchmarks: FaceForensics 256^2 [55], SkyTimelapse 256^2 [80], UCF101 256^2 [64], RainbowJelly 256^2 (introduced in our work) and MEAD 1024^2 [74]. Apart from our model, we train from scratch 5 different methods and measure their performance using the same evaluation protocol. Frechet Video Distance (FVD) [70] serves as the main metric for video synthesis, but there is no *complete* official implementation for it (see Sec 4 and Appx C). This leads to discrepancies in the evaluation procedures used by different works because FVD, similarly to FID [23], is *very* sensitive to data format and sampling strategy [48]. That’s why we implement, document and release our complete FVD evaluation protocol. In terms of sheer metrics, our method performs on average $\approx 30\%$ better than the closest runner-up.

2. Related work

Video synthesis. Early works on video synthesis mainly focused on *video prediction* [36, 72], i.e. generating future frames given a sequence of the previously seen ones. Early approaches for this problem typically employed recurrent convolutional models trained with reconstruction objective [17, 54, 65], but later adversarial losses were introduced to improve the synthesis quality [37, 71, 75]. Some recent works explore autoregressive video prediction with recurrent or attention-based models (e.g., [27, 53, 73, 78, 81]). Another close line of research is *video interpolation*, i.e. increasing the frame rate of a given video (e.g., [6, 25, 44]). In our work, we study *video generation*, which is a more challenging problem than video prediction since it seeks to synthesize videos from scratch, i.e. without using the expressive conditioning on previous frames. Classical methods in this direction are typically based on GANs [20].

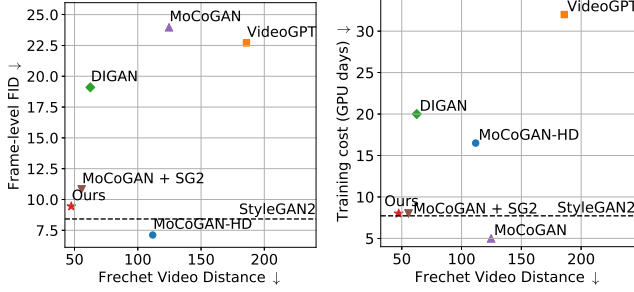


Figure 3. FID scores and training cost by FVD₁₆ for modern video generators on FaceForensics 256² [55]. Our method (denoted by $\star$) shows that video generators can be as efficient and as good in terms of image quality as traditional image based generators (like, StyleGAN2 [31], denoted with the dashed line). FID for video generators was computed from 16-frames long videos.

MoCoGAN [69] and TGAN [56] decompose generator’s input noise into a content code and motion codes, which became a standard strategy for many subsequent works (e.g., [3, 42, 57, 67]). SVGAN [24] additionally add self-supervision losses to improve the synthesis.

MoCoGAN-HD [67] and StyleVideoGAN [18], similar to us, consider high-resolution video synthesis. But in their case, the authors perform *indirect* training by training a motion codes generator in the latent space of a pretrained StyleGAN2 model. StyleGAN-V is trained on *extremely* sparse videos. This makes it related to [10, 57, 77], which use a pyramid of discriminators operating on different temporal resolutions (with a subsampling factor of up to $\times 8$). In contrast to the prior work, our generator is continuous in time. In this way it is similar to Vid-ODE [47]: a continuous-time video interpolation and prediction model based on neural ODEs [13].

To the best of our knowledge, all modern video synthesis approaches utilize expensive `conv3d` blocks either in their decoder and/or encoder components (e.g., [2, 14, 26, 42, 57, 67, 69]). Often, GAN-based approaches utilize two discriminators, operating on image and video levels independently, where the video discriminator operates at a low resolution to save computation (e.g., [14, 67, 69, 76]). In our work, we show that it’s enough to use a single holistic hypernetwork-based [21] discriminator conditioned on the time difference between frames to build a state-of-the-art temporarily coherent video generator. For conditioning, we use `ModConv2d` blocks from [31], which is similar to AnyCostGAN [34].

Neural Representations. Neural representations is a recent paradigm that uses neural networks to represent continuous signals, such as images, videos, audios, 3D objects and scenes (e.g., [19, 40, 60, 61, 66]). It is mostly popular for 3D reconstruction and geometry processing tasks (e.g., [35, 39, 43, 45, 50]), including video-based reconstruc-

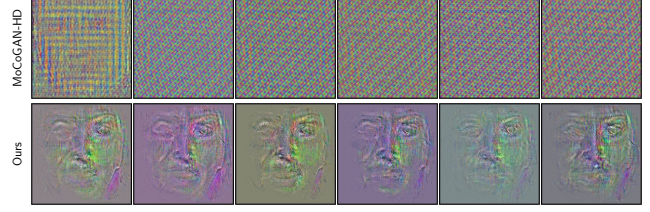


Figure 4. Visualizing the gradient signal to G at $\approx 50\%$ of training from `conv3d`-based discriminator of MoCoGAN-HD (upper row) and our hypernetwork-based D (lower row) at $t = 0, 2, 4, 6, 8, 12$ timesteps.

tion [33, 46, 51, 79]. Several recent projects explored the task of building generative models over such representations to synthesize images (e.g., [4, 62, 63]), 3D objects (e.g., [11, 32, 58]) or multi-modal signals (e.g., [15, 16]), and our work extends this line of research to video generation.

Concurrent works. The development of neural representations-based approaches moves extremely fast and there are two concurrent works which propose ideas similar to our ones. DIGAN [5] is a concurrent project that explores the same direction of using neural-based representations for continuous video synthesis and shares a *lot* of ideas with our work. The authors also consider a continuous-time generator, trained by a discriminator without `conv3d` layers. The core difference with our work is that they use a different parametrization of motions and use a dual discriminator D: one operates on $(x_1, x_2, \Delta t)$ and the second one on individual images. We enumerate the differences and similarities in Appx H. NeRV [12] is another concurrent project which proposes to represent videos as convolutional neural representations. But in their case, the authors explore compression and denoising tasks. GEM [15] utilizes generative latent optimization [7] to build a multi-modal generative model.

3. Model

Our model is based on the paradigm of *neural representations* [40, 60, 66], i.e. representing signals as neural networks. We treat each video as a function $x_t = x(t)$ which is continuous in time $t \in \mathbb{R}_+$. In this manner, the training dataset $\mathcal{D}$ is a set of subsampled signals $\mathcal{D} = \{x^{(i)}\}_{i=1}^N = \{(x_{t_0}^{(i)}, \dots, x_{t_{\ell_i}}^{(i)})\}_{i=1}^N$, where N denotes the total number of videos, t_j denotes the time position of the j -th frame and ℓ_i is the amount of frames in the i -th video³. Note that each video might have a different length ℓ_i and in practice these lengths vary a lot (see Appx E for datasets statistics). Our goal is to train a generative model over video signals, having only their subsampled versions. To achieve this, we develop

³To simplify the notation, we assume that all videos have the same frame-rate and that all the videos were sampled starting at $t = t_0$, but it is not a limitation of the method

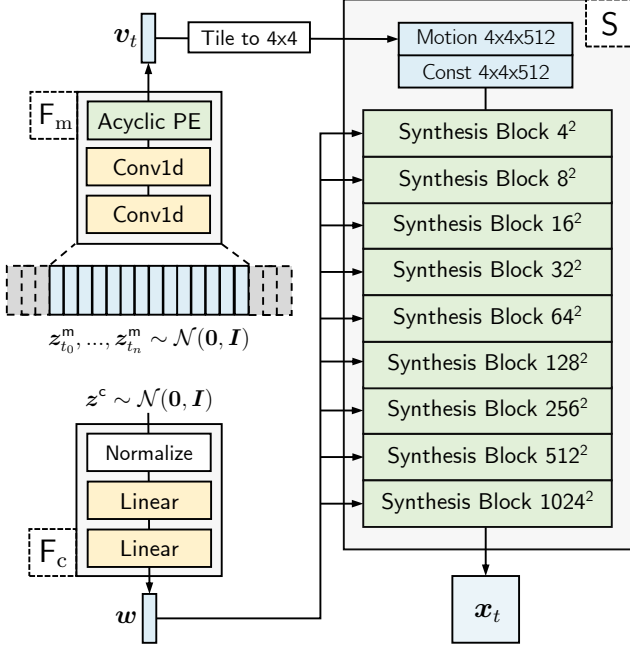


Figure 5. Our generator architecture: the only change we do on top of StyleGAN2 generator’s synthesis network S is the concatenation of our motion codes to the constant input tensor. S produces frames x_t independently from each other using the content code w and motion code v_t . Since F_m is non-autoregressive, our video generator is also non-autoregressive. Synthesis blocks are utilized without any changes from StyleGAN2 [31]. For 256²-resolution datasets, we remove the last two blocks from S .

the following framework.

We build the model on top of StyleGAN2 [29] and redesign its generator and discriminator networks for video synthesis. Our generator is conceptually similar to MoCoGAN [69], i.e., we separate latent information into content code z^c and motion trajectory $v_t = v(t)$. In contrast to MoCoGAN, our motion codes v_t are continuous in time $t \in \mathbb{R}_+$ and we describe their design in Sec 3.1. The *only* modification we do on top of StyleGAN2’s generator is the concatenation of our continuous motion codes v_t to its constant input tensor. In all other aspects, it is *entirely equivalent* to its image-based counterpart.

The discriminator model D takes k frames $x_{t_1}, \dots, x_{t_k}$ of a sparsely sampled video, independently extracts features $h_{t_1}, \dots, h_{t_k}$ from them, concatenates those features together channel-wise into a global video descriptor h and predicts the real/fake class from it. D is conditioned on the time distances $\delta_i^x = t_{i+1} - t_i$ between frames and we use hypernetwork-based conditioning to input this information.

3.1. Generator structure

Overview. Generator consists of three components: content mapping network F_c , motion mapping network F_m

and synthesis network S . F_c and S are equivalent to their StyleGAN2’s counterparts with the exception that we tile and concatenate motion codes v_t to the constant input tensor of S .

A video is generated the following way. First, we sample the content noise $z^c \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and, following StyleGAN2, transform it into latent code $w = F_c(z^c) \in \mathbb{R}^{512}$. It is shared for all timesteps $t \in \mathbb{R}_+$ of a video. Then, to generate a frame x_t in the specified time location t , we first compute its motion code v_t , which is done in three steps. First, we sample a discrete sequence of equidistant trajectory noise $z_{t_0}^m, \dots, z_{t_n}^m \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ (we assume $t_0 = 0$ everywhere), positioned at distance $\delta^z = t_{i+1} - t_i$ from one another. The number of tokens n is determined by the condition $t < t_n$, i.e. it should be long enough to cover the desired timestep t .⁴ Then, we process it with conv1d-based motion mapping network F_m with a large kernel size into the sequence $u_{t_0}, \dots, u_{t_n}$. After that, we take a pair of tokens u_ℓ, u_r which t lies between (i.e. $\ell = t_i$ for some $i \in \{0, 1, \dots, n\}$ and $r = t_{i+1}$) and compute an acyclic positional embedding v_t from them, described next. This positional embedding serves as the motion code for our generator. In fact, we do not need to sample all the motion noise vectors $z_{t_0}^m, \dots, z_{t_n}^m$ to produce v_t , but only those ones which v_t depends on. In this way, our generator can produce frames non-autoregressively.

Acyclic positional encoding. Traditional positional embeddings [60, 66] are cyclic by default. This does not create problems in traditional applications (like image or scene representations) because utilized spatial domain there never exceeds the period length [40, 62]. But for video generation, cyclicity is not desirable, because it makes a video getting looped at some point. To solve this issue, we develop the *acyclic* positional encoding mechanism.

A sine-based positional embedding vector $p \in \mathbb{R}^d$ can be expressed in the following form:

$$p(\alpha, \omega, \rho, t) = \alpha \odot \sin(\omega \cdot t + \rho), \quad (1)$$

where $\odot$ denotes element-wise vector multiplication, $\alpha, \omega, \rho \in \mathbb{R}^d$ are amplitudes, periods and phases of the corresponding waves, and the sine function is applied element-wise. By default, these embeddings are periodic and always the same for any input [40, 60, 66], which is not desirable for video synthesis, where natural videos contain different motions and are typically aperiodic. To solve this issue, we compute the wave parameters from motion noise $z_{t_0}^m, \dots, z_{t_n}^m$ the following way. First, “raw” motion codes $\tilde{v}_t$ are computed using wave parameters $\alpha_\ell, \omega_\ell, \rho_\ell$ predicted from u_ℓ :

$$\tilde{v}_t = \alpha_\ell \odot \sin(\omega_\ell \cdot t + \rho_\ell), \quad (2)$$

⁴In practice, since F_m uses padding-less convolutions, this sequence is slightly larger. We elaborate on this in Appx B.

where

$$\alpha_\ell = W_\alpha \mathbf{u}_\ell, \quad \omega_\ell = W_\omega \mathbf{u}_\ell, \quad \rho_\ell = W_\rho \mathbf{u}_\ell, \quad (3)$$

and $W_\alpha, W_\omega, W_\rho \in \mathbb{R}^{d \times d}$ are learnable weight matrices. Using $\tilde{\mathbf{v}}_t$ directly as motion codes does not lead to good results since it contains discontinuities (see Fig 9d). That’s why we “stitch” their start and end values via:

$$\mathbf{v}_t = \tilde{\mathbf{v}}_t - \text{lerp}(\tilde{\mathbf{v}}_\ell, \tilde{\mathbf{v}}_r, t) + \text{lerp}(W_a \mathbf{u}_\ell, W_a \mathbf{u}_r, t), \quad (4)$$

where $W_a \in \mathbb{R}^{d \times d}$ is a learnable weight matrix and $\text{lerp}(\mathbf{x}, \mathbf{y}, t)$ is the element-wise linear interpolation between $\mathbf{x}$ and $\mathbf{y}$ using the time position t . The first subtraction in Eq (4) alters the positional embeddings to make them converge to zero values at locations $\{t_0, t_1, \dots, t_n, \dots\}$. This limits the expressive power of the positional embeddings and that’s why we add the “alignment” vectors $\mathbf{a} = W_a \mathbf{u}$ to restore it. See Fig 9e for the visualization.

In practice, we found it useful to compute periods as:

$$\omega_t = (\tanh(W_\omega \mathbf{u}_t) + \mathbb{1}) \odot \sigma, \quad (5)$$

where $\mathbb{1}$ is a vector of ones and σ are linearly-spaced scaling coefficients. See Appx B and the source code for details.

One could try using continuous codes $\mathbf{u}_t = \text{lerp}(\mathbf{u}_\ell, \mathbf{u}_r, t)$ directly as motion codes instead of $\mathbf{v}_t$. This also eliminates cyclicity (in theory), but leads to poor results in practice: if the distance δ^z is small, then the motion trajectory will contain unnatural sharp transitions; and when δ^z is increased, G loses its ability to properly model high-frequency motions (like blinking) since the codes change too slowly. We empirically validate this Tab 2 (also see samples on the project webpage).

3.2. Discriminator structure

Modern video generators typically utilize two separate discriminators which operate on image and video levels separately [14, 67, 69]. But since we train on extremely sparse videos and aim to have a computationally efficient model, we propose to use a *holistic* hypernetwork-based discriminator $D(\mathbf{x}_{t_1}, \dots, \mathbf{x}_{t_k})$, which is conditioned on the time distances between frames $\delta_i^x = t_{i+1} - t_i$. It consists of two parts: 1) feature extractor backbone D_b , which independently embeds an image frame $\mathbf{x}_t$ into a 3D feature vector $\mathbf{h}_{t_i} \in \mathbb{R}^{512 \times 16 \times 16}$; and the convolutional head D_h , which takes the concatenation of all the features $\mathbf{h} = \text{concat}[\mathbf{h}_{t_1}, \dots, \mathbf{h}_{t_k}] \in \mathbb{R}^{512k \times 16 \times 16}$ and outputs the real/fake logit $y \in \mathbb{R}$.

We input the time distances information $\delta_1^x, \dots, \delta_{k-1}^x$ between k frames $\mathbf{x}_{t_1}, \dots, \mathbf{x}_{t_k}$ into D the following way. First, we encode them with positional encoding [60, 66], preprocess with a 2-layer MLP into $\mathbf{p}(\delta_1^x), \dots, \mathbf{p}(\delta_{k-1}^x) \in \mathbb{R}^d$ and concatenate into a single vector $\mathbf{p}_\delta \in \mathbb{R}^{(k-1)d}$. After that,

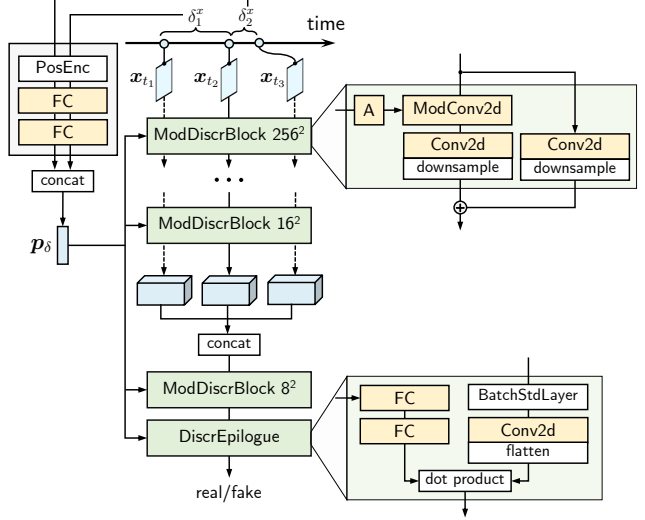


Figure 6. Discriminator architecture for $k = 3$ frames per video. Traditional StyleGAN2’s `DiscrBlock` [31] is replaced with `ModDiscrBlock` to use `ModConv2d`. `DiscrEpilogue` is equivalent to StyleGAN2’s implementation: we refer an interested reader to [31] for details on it. `ModConv2d` is the `Conv2d` layer with weight demodulation from StyleGAN2’s generator. The discriminator is conditioned on δ_i^x in two ways: via `ModConv2d` and via the projection [41] in `DiscrEpilogue` (the default conditioning scheme of StyleGAN2).

we use $\mathbf{p}_\delta$ to modulate the weights of each first convolutional layer in each block of D_b and D_h and also as a conditional vector in the projection head [41] in the StyleGAN2’s `DiscrEpilogue` block. The modulation in D is equivalent to modulation in G, but uses $\mathbf{p}_\delta$ instead of $\mathbf{w}$ vector: it is passed through a mapping network, transformed with an affine layer and multiplied on a 4D weight tensor of a convolutional layer across the input channel axis. We do not modulate *each* convolutional layer from practical considerations: in practice, `ModConv2d` is $\approx 25\%$ heavier than `Conv2d`. The overall architecture is visualized in Fig 6.

Such an approach is *greatly* more efficient than using dual discriminators D_I and D_V since we no longer need the expensive `conv3d`-based discriminator, which is too expensive to operate on high-resolution videos [14]. Moreover, as we demonstrate in Fig 4, it provides a more informative learning signal to G.

3.3. Implicit assumptions of sparse training

Videos are continuous signals and any practical video generator relies on some sort of subsampling. The question of how many samples are necessary to train a video synthesis model is fundamental, because this design decision greatly influences the quality and training cost. In our work, we empirically show that one can train a state-of-the-art video generator with as few as 2 frames per video.



Figure 7. Random samples from the existing methods on FaceForensics 256^2 , SkyTimelapse 256^2 and RainbowJelly 256^2 , respectively. We sample a 64-frames video and display each 4-th frame, starting from $t = 0$.

Consider the problem of learning a probability distribution $p(\mathbf{x}) = p(x_1, \dots, x_n)$ and consider that we utilize sparse training, i.e. select k coordinates of the vector $\mathbf{x}$ randomly on each iteration of the optimization process. Then our optimization objective is equivalent to learning all possible marginal distributions $p(x_{i_1}, \dots, x_{i_k})$ instead of learning the joint distribution $p(\mathbf{x})$. When does learning marginals allow to obtain the full joint distribution at the end? The following simple statement adds some clarity to this question.

A trivial but serviceable statement. Let's denote by $\mathcal{J}_{< i}^k$ a collection of sets J_i of up to k indices j s.t. $\forall J_i \in \mathcal{J}_{< i}^k$ we have $j < i$ for all $j \in J_i$. In other words, J_i is a set of up to k indices $j \in [1, i)$. Then, $p(\mathbf{x})$ can be represented as a product of n marginals $p(x_i, \mathbf{x}_{J_i})$ for $i \in [1, n]$ if and only if $\forall i$ there exists $J_i \in \mathcal{J}_{< i}^{k-1}$ s.t. $p(x_i | \mathbf{x}_{< i}) \equiv p(x_i | \mathbf{x}_{J_i})$.

The above statement is primitive (see the proof in Appx F) but can provide useful practical intuition. For video synthesis, it implies that one can learn a video generator by using only k frames per video only if for any frame $\mathbf{x}_i$, there exists at most $k - 1$ previous frames sufficient to properly predict it (see Appx F). And we argue that for the modern video synthesis datasets, one does not need a lot of frames to make such a prediction. For example, for SkyTimelapse [80], the motions are typically unidirectional and

thus easily predictable from only 2 previous frames (which corresponds to training with $k = 3$ frames per video). But surprisingly, in practice we found that using even $k = 2$ frames per clip can provide state-of-the-art performance.

We treat videos as infinite continuous signals, but in practice one has to set a limit on the maximum time location T which can be seen during training. To the best of our knowledge, previous methods use at most $T = 64$ [10, 57], but in our case we train the model with $T = 1024$, which does not lead to much additional computational burden due to the non-autoregressive nature of our generator. However, we set the maximum distance between t_1 and t_k to 32 to cover short-term and medium-term movements: otherwise, we observed unstable training and abrupt motions in video samples. To sample frames, we first sample the distance $(t_k - t_1) \sim U[k - 1, 32]$ and then $t_1 \sim U[0, T - t_k]$. After that, frames locations t_i for $i \in \{2, \dots, k - 1\}$ are selected at random without repetitions.

4. Experiments

Datasets. We test our model on 5 benchmarks: FaceForensics 256^2 [55], SkyTimelapse 256^2 [80], UCF101 256^2 [64], RainbowJelly 256^2 (introduced by us) and MEAD 1024^2 [74]. We used the train splits (when they are available) for all the datasets except for UCF101, which

Table 1. Quantitative performance and training cost of different methods. We trained *all* the methods from scratch on 256^2 resolution datasets using the official codebases and evaluated them under the unified evaluation protocol (see Sec 4). Training was done on $\times 4$ 32 GB NVidia V100 GPUs for all the methods except VideoGPT, which was trained on $\times 4$ NVidia A6000 GPUs (with 48.5 GB of memory each) due to its high memory consumption. For 2-stage methods, we report their training cost in the “ $X + Y$ ” format. [†]VideoGPT was trained for our maximum resource constraint of 32 GPU-days which was detrimental to its performance on 256^2 resolution. Vanilla StyleGAN2 training time on 256^2 resolution (with mixed precision and optimizations [29]) is 7.72 GPU-days in our environment.

Method	FaceForensics		SkyTimelapse		UCF101		RainbowJelly		Training cost (GPU-days)
	FVD ₁₆	FVD ₁₂₈	FVD ₁₆	FVD ₁₂₈	FVD ₁₆	FVD ₁₂₈	FVD ₁₆	FVD ₁₂₈	
MoCoGAN [69]	124.7	257.3	206.6	575.9	2886.9	3679.0	1572.9	549.7	5
+ StyleGAN2 backbone	55.62	309.3	85.88	272.8	1821.4	2311.3	638.5	463.0	8
MoCoGAN-HD [67]	111.8	653.0	164.1	878.1	1729.6	2606.5	579.1	628.2	7.5 + 9
VideoGPT [81] [†]	185.9	N/A	222.7	N/A	2880.6	N/A	136.0	N/A	16 + 16
DIGAN [5] (concurrent work)	62.5	1824.7	83.11	196.7	1630.2	2293.7	436.6	369.0	16
StyleGAN-V (ours)	47.41	89.34	79.52	197.0	1431.0	1773.4	195.4	262.5	8

is an extremely difficult dataset and we used `train+test` splits for it. We noticed that modern video synthesis datasets are either too simple or too difficult in terms of content and motion, and there are no datasets “in-between”. That’s why we introduce RainbowJelly: a dataset of “floating” jellyfish from the Hoccori Japan youtube channel, which contains 8 hours of videos in total and has $>1024^2$ resolution. It contains simple content but complex hierarchical motions and this makes it a challenging but approachable test-bed for evaluating modern video generators. We provide its details in Appx E. All the datasets have 25 FPS, except for RainbowJelly and MEAD, which have 30 FPS.

Evaluation. Following prior work, we use Frechet Video Distance (FVD) [70] and Inception Score (IS) as our evaluation metrics with FVD being the main one since FID (its image-based counterpart) better aligns with human-perceived quality [23]. We use two versions of FVD: FVD₁₆ and FVD₁₂₈, which use 16-frames-long and 128-frames-long videos respectively to compute their statistics. Inception Score is used only to evaluate the generation quality on UCF-101 since it uses a UCF-101-finetuned C3D model [56].

The official FVD project [70] does not provide the *complete* implementation of the evaluation pipeline, but rather the inference script for a single batch of videos, which are required to be *already resized to 256^2 and loaded into memory*. This creates discrepancies in the evaluation protocols used by previous works since FVD, (similar to FID [48]) is very sensitive to the interpolation procedures and perceptually unnoticeable artifacts introduced by data processing procedures, like JPEG compression. We also found it to be very sensitive to how one extracts clips from real videos to compute the statistics. We implement and release a complete FVD evaluation protocol and use it to evaluate all the methods for fair comparison. It is documented in Appx C.

Baselines. We use 5 baselines for comparison: MoCoGAN [69], MoCoGAN [69] with the StyleGAN2 [31]

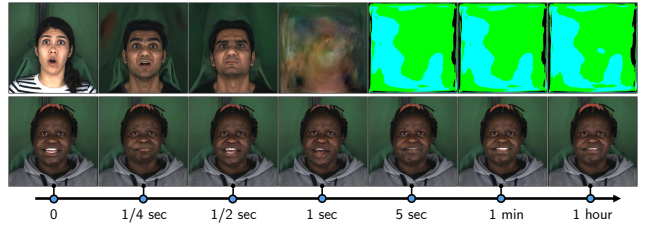


Figure 8. Generations on MEAD 1024^2 [74] for MoCoGAN-HD [67] and our method. MoCoGAN-HD cannot preserve the identity and diverges for long LSTM unrolling. (Note that all videos in the dataset have static head positions — see Appx E).

backbone, VideoGPT [81], MoCoGAN-HD [67] and DIGAN [5]. For MoCoGAN with the StyleGAN2 backbone (denoted as MoCoGAN-SG2), we replaced its generator and image-based discriminator with the corresponding StyleGAN2’s components, leaving its video discriminator unchanged. We also used the training scheme and regularizations from StyleGAN2. MoCoGAN was trained for 5 days on a single GPU since its lightweight DC-GAN [52] backbone makes it fast to train, while MoCoGAN+SG2 was trained for 2 days on $\times 4$ GPUs to reach 25M real images seen by its image-based discriminator. MoCoGAN-HD is trained for ≈ 4.5 days on $\times 4$ v100 GPUs, as specified in the original paper (Appx B of [67]). We trained VideoGPT for the maximum affordable total time of 32 GPU-days in our resource constraints. We trained DIGAN [5] for 5 days since we observed that by that time the metrics either plateaued or exploded (for RainbowJelly). We also noticed that DIGAN uses weighted sampling during training by selecting clips from long videos with higher probabilities. This hurts its FVD score and that’s why we altered its data sampling strategy to the uniform one, which is used by other methods [67, 69, 81]. For each method we selected the checkpoint with the best FVD₁₆ value.

Table 2. Ablating architectural components of our model.

Method	FaceForensics 256 ²		SkyTimelapse 256 ²	
	FVD ₁₆	FVD ₁₂₈	FVD ₁₆	FVD ₁₂₈
Default StyleGAN-V	47.41	89.34	79.52	197.0
w/o our G	65.88	41.77	109.1	240.2
w/o our D	154.0	139.1	236.9	258.0
w/o hyper-modulation in D	88.8	161.5	69.8	184.1
w/o projection cond in D	95.4	236.0	102.1	210.3
w LSTM codes, $\delta^z = 1$	131.9	159.1	135.7	196.1
w LSTM codes, $\delta^z = 16$	180.3	94.55	95.71	165.8

4.1. Main experiments

For the main evaluation, we train our method and all the baselines from scratch on the described 256² datasets. Each model is trained on $\times 4$ NVidia V100 32 GB GPUs, except for VideoGPT, which is very demanding in terms of GPU memory for 256² resolution and we had to train it on $\times 4$ NVidia A6000 instead (with the overall batch size of 4). For our method and MoCoGAN+SG2, we use exactly the same optimization scheme as StyleGAN2, including the loss function, Adam optimizer hyperparameters and R1 regularization [38]. We reduce the learning rate by 10 for the D_V module of MoCoGAN+SG2 since it does not have equalized learning rate [28]. We use $\delta^z = 16$ for all the experiments except for SkyTimelapse, where we used $\delta^z = 256$. See other training details in Appx B. We evaluate all the methods under the same evaluation protocol, described in Appx C and report the results in Table 1.

To measure the efficiency, we use the amount of GPU days required to train a method. We build on top of the official StyleGAN2 implementation.⁵ The training cost of the image-based StyleGAN2 to reach its specified 25M images is 7.72 NVidia V100 GPU-days in our environment. StyleGAN-V is trained for 2 days, which corresponds to ≈ 23 M real frames seen by the discriminator. MoCoGAN-HD is built on top of `stylegan2-pytorch`'s code-base⁶, which is ≈ 2 times slower than the highly optimized NVidia's implementation. That's why in Table 1 we report its training cost *reduced* by a factor of 2 to account for this.

Our method significantly outperforms the existing ones at almost all the benchmarks in terms FVD₁₆ and FVD₁₂₈. On UCF101, all the methods perform poorly, which shows that it is a too difficult benchmark for modern video generators and one needs to train models of extreme scale to fit it [14]. For completeness, the Inception Score [56] results are provided in Table 5 in Appx C.

We visualize the samples in Fig 1 and Fig 7. Our method is able to generate hour-long plausibly looking videos, though the motion diversity and global motion coherence

⁵<https://github.com/NVlabs/stylegan2-ada-pytorch>

⁶<https://github.com/rosinality/stylegan2-pytorch>

Table 3. Ablating the amount of frames k per clip used during training. Sparse training provides better results for our method.

Number of frames	FaceForensics 256 ²		SkyTimelapse 256 ²	
	FVD ₁₆	FVD ₁₂₈	FVD ₁₆	FVD ₁₂₈
$k = 2$	60.41	93.5	50.5	209.9
$k = 3$ (default)	47.41	89.34	79.52	197.0
$k = 4$	51.84	114.9	65.7	194.5
$k = 8$	101.9	211.4	73.12	215.9
$k = 16$	92.52	192.8	107.6	254.3

for them would be limited (see Appx A). MoCoGAN-HD suffers from the LSTM instability when unrolled to large lengths and does not produce diverse motions. DIGAN produces high-quality videos on SkyTimelapse because its inductive bias of having joint spatio-temporal positional information is well suited for videos that have an entire scene moving. But for FaceForensics, this leads to a "head flying away" effect (see Appx H). To generate 1-hour long videos from MoCoGAN-HD, we unroll its LSTM model to the required depth ($\approx 90k$ steps) and synthesize frames only in the necessary time positions, while DIGAN, similar to our method, is able to generate frames non-autoregressively.

4.2. Ablations

To ablate the core components, we replaced G or D modules with their MoCoGAN+SG2 counterparts. In the both cases, their removal leads to poor short-term and long-term video quality, as specified by the corresponding metrics in Table 2 and video samples in the supplementary.

Replacing continuous motion codes $v(t)$ with $u(t)$, produced by the LSTM model hurts the performance, especially when the distance δ^z between motion codes is small. This happens due to unnaturally abrupt transitions between frames and we provide the corresponding samples in the supplementary. The corresponding results are in Table 2.

Another architectural decision which we verify is the conditioning scheme utilized in D. We use both `ModConv2d` blocks and the projection conditioning head [41] in `DiscrEpilogue` in our discriminator to provide the conditioning signal about the time differences between frames $\delta_1^x, \dots, \delta_{k-1}^x$. Removing any of them hurts the performance, because it constrains the ability of D to understand the temporal scale it is currently operating on. We ablated the hypernetwork-based modulation by feeding a vector of zeros instead of positional embeddings of time differences into `ModConv2d` so that only the bias vectors in the corresponding affine transforms participate in the modulation process.

An important design choice is how many samples per video one should use during training. We try different values of k for $k = 2, 3, 4, 8$ and 16 and report the corresponding results in Table 3. As being discussed in Sec 3.3, for

existing video generation benchmarks, it might be enough to sample only several frames per each video, and our experiments confirm this observation. The performance is decreased for larger k , but this might be attributed to a weaker temporal aggregation procedure of D, which simply concatenates features together. It is surprising to see that modern datasets can be fit with as few as 2 samples per video.

4.3. Properties

Our generator is able to generate arbitrarily long videos. Our design of motion codes allows StyleGAN-V not to suffer from stability problems when unrolled to large (potentially infinite) video lengths. This is verified by visualizing the video clips for the extremely large timesteps in Fig 1 and Fig 8. We also demonstrate its ability to produce videos in arbitrarily high frame-rate in the supplementary.

Our model has the same latent space manipulation properties as StyleGAN2. To show this, we conduct two experiments: embedding, editing and animating an off-the-shelf image and editing and animating the first frame of a generated video. To embed an image, we used the optimization procedure similar to [1], but considering it to be positioned at $t = 0$. To edit an image with CLIP, we used the procedure of [49]. The results of these experiments are visualized in Fig 2 and we provide the details in Appx B and more examples in the supplementary. Apart from showing the good properties of its latent space, these experiments demonstrate the extrapolation potential of our generator.

StyleGAN-V has almost the same training efficiency and image quality as StyleGAN2. In Fig 3, we plot the FID scores and training costs of modern video generators on FaceForensics 256² by their corresponding FVD₁₆ scores. Our method comes very close to StyleGAN2: it converges to FID of 9.44 in 8 GPU-days compared to FID of 8.42 in 7.72 GPU-days for StyleGAN2, which is only $\approx 10\%$ worse. This raises a question of whether video generators can be as computationally efficient and good in terms of image quality as image-based ones.

Our model is the first one which is directly trainable on 1024² resolution. We provide the generations on MEAD 1024² for our method and for MoCoGAN-HD. MoCoGAN-HD cannot preserve the identity of a speaker and diverges for large video lengths, while our method achieves comparable image quality and coherent motions. For this dataset, our model was trained for 7 days on $\times 4$ NVidia v100 GPUs and obtained FID of 24.12 and FVD₁₆ of 156.1. Image generator for MoCoGAN-HD was trained for 14 days on $\times 4$ A6000 GPUs, while its video generator was trained for only 5 days since it didn't require high-resolution training.

Our discriminator provides more informative learning signal to G. Fig 4 visualizes the gradient signal to the generator from our discriminator and the conv3d-based video discriminator of MoCoGAN-HD, measured at $\approx 50\%$

of training for our method (at 10M images seen by D) and MoCoGAN-HD (at the 300-th epoch). In our case, one can easily see fine-grained details of the face structure, perceived by D, while in case of MoCoGAN-HD, most of the gradient is redundant and lack any structural information.

Content and motion decomposition. Similar to MoCoGAN [69], our generator captures content and motion variations in a disentangled manner: altering motion codes $z_{t_0}^m, \dots, z_{t_n}^m$ while fixing z^c does not change the appearance variations (like, a speaker's identity). Similarly, re-sampling z^c does not influence motion patterns on a video, but only its content. We provide the corresponding visualizations on the project website.

5. Conclusion

In this work, we provided a different perspective on time for video synthesis and built a continuous video generator using the paradigm of neural representations. For this, we developed motion representations through the lens of positional embeddings, explored sparse training of video generators and redesigned a typical dual structure of a video discriminator. Our model is built on top of StyleGAN2 and features a lot of its perks, like efficient training, good image quality and editable latent space.

We hope that our work would serve as a solid basis for building more powerful video generators in the future. The limitations and potential negative impact are discussed in Appx A.

References

- [1] Rameen Abdal, Yipeng Qin, and Peter Wonka. Image2stylegan: How to embed images into the stylegan latent space? In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4432–4441, 2019. 9, 13, 14
- [2] Dinesh Acharya, Zhiwu Huang, Danda Pani Paudel, and Luc Van Gool. Towards high resolution video generation with progressive growing of sliced wasserstein gans. *arXiv preprint arXiv:1810.02419*, 2018. 3
- [3] Abhishek Aich, Akash Gupta, Rameswar Panda, Rakib Hyder, M Salman Asif, and Amit K Roy-Chowdhury. Non-adversarial video synthesis with learned priors. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6090–6099, 2020. 2, 3
- [4] Ivan Anokhin, Kirill Demochkin, Taras Khakhulin, Gleb Sterkin, Victor Lempitsky, and Denis Korzhnikov. Image generators with conditionally-independent pixel synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14278–14287, 2021. 3, 14
- [5] Anonymous. Generating videos with dynamics-aware implicit generative adversarial networks. In *Submitted to The Tenth International Conference on Learning Representations*, 2022. under review. 1, 3, 7, 13, 16, 19

- [6] Wenbo Bao, Wei-Sheng Lai, Chao Ma, Xiaoyun Zhang, Zhiyong Gao, and Ming-Hsuan Yang. Depth-aware video frame interpolation. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2019. 2
- [7] Piotr Bojanowski, Armand Joulin, David Lopez-Paz, and Arthur Szlam. Optimizing the latent space of generative networks. *arXiv preprint arXiv:1707.05776*, 2017. 3
- [8] Andrew Brock, Jeff Donahue, and Karen Simonyan. Large scale gan training for high fidelity natural image synthesis. *arXiv preprint arXiv:1809.11096*, 2018. 1
- [9] Joao Carreira and Andrew Zisserman. Quo vadis, action recognition? a new model and the kinetics dataset. In *proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 6299–6308, 2017. 14
- [10] Lluís Castrejon, Nicolas Ballas, and Aaron Courville. Hierarchical video generation for complex data. *arXiv preprint arXiv:2106.02719*, 2021. 3, 6
- [11] Eric R Chan, Marco Monteiro, Petr Kellnhofer, Jiajun Wu, and Gordon Wetzstein. pi-gan: Periodic implicit generative adversarial networks for 3d-aware image synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5799–5809, 2021. 3
- [12] Hao Chen, Bo He, Hanyu Wang, Yixuan Ren, Ser-Nam Lim, and Abhinav Shrivastava. Nerv: Neural representations for videos. *arXiv preprint arXiv:2110.13903*, 2021. 3
- [13] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018. 3
- [14] Aidan Clark, Jeff Donahue, and Karen Simonyan. Adversarial video generation on complex datasets. *arXiv preprint arXiv:1907.06571*, 2019. 1, 2, 3, 5, 8
- [15] Yilun Du, Katherine M. Collins, Joshua B. Tenenbaum, and Vincent Sitzmann. Learning signal-agnostic implicit manifolds. In *Thirty-Fifth Conference on Neural Information Processing Systems*, 2021. 3
- [16] Emilien Dupont, Yee Whye Teh, and Arnaud Doucet. Generative models as distributions of functions. *arXiv preprint arXiv:2102.04776*, 2021. 3
- [17] Chelsea Finn, Ian Goodfellow, and Sergey Levine. Unsupervised learning for physical interaction through video prediction. *Advances in neural information processing systems*, 29:64–72, 2016. 2
- [18] Gereon Fox, Ayush Tewari, Mohamed Elgharib, and Christian Theobalt. Stylevideogan: A temporal generative model using a pretrained stylegan. *arXiv preprint arXiv:2107.07224*, 2021. 3
- [19] Kyle Genova, Forrester Cole, Daniel Vlasic, Aaron Sarna, William T Freeman, and Thomas Funkhouser. Learning shape templates with structured implicit functions. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 7154–7164, 2019. 3
- [20] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. *Advances in neural information processing systems*, 27, 2014. 2
- [21] David Ha, Andrew Dai, and Quoc V Le. Hypernetworks. *arXiv preprint arXiv:1609.09106*, 2016. 2, 3
- [22] Amir Hertz, Or Perel, Raja Giryes, Olga Sorkine-hornung, and Daniel Cohen-or. SAPE: Spatially-adaptive progressive encoding for neural optimization. In *Thirty-Fifth Conference on Neural Information Processing Systems*, 2021. 14, 17
- [23] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. *Advances in neural information processing systems*, 30, 2017. 2, 7
- [24] Sangeek Hyun, Jihwan Kim, and Jae-Pil Heo. Self-supervised video gans: Learning for appearance consistency and motion coherency. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10826–10835, June 2021. 3
- [25] Huaizu Jiang, Deqing Sun, Varun Jampani, Ming-Hsuan Yang, Erik Learned-Miller, and Jan Kautz. Super slo-mo: High quality estimation of multiple intermediate frames for video interpolation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 9000–9008, 2018. 2
- [26] Emmanuel Kahembwe and Subramanian Ramamoorthy. Lower dimensional kernels for video discriminators. *Neural Networks*, 132:506–520, 2020. 3
- [27] Nal Kalchbrenner, Aaron Oord, Karen Simonyan, Ivo Danihelka, Oriol Vinyals, Alex Graves, and Koray Kavukcuoglu. Video pixel networks. In *International Conference on Machine Learning*, pages 1771–1779. PMLR, 2017. 2
- [28] Tero Karras, Timo Aila, Samuli Laine, and Jaakko Lehtinen. Progressive growing of gans for improved quality, stability, and variation. *arXiv preprint arXiv:1710.10196*, 2017. 8, 13
- [29] Tero Karras, Miika Aittala, Janne Hellsten, Samuli Laine, Jaakko Lehtinen, and Timo Aila. Training generative adversarial networks with limited data. *arXiv preprint arXiv:2006.06676*, 2020. 1, 4, 7, 13, 14, 16
- [30] Tero Karras, Samuli Laine, and Timo Aila. A style-based generator architecture for generative adversarial networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4401–4410, 2019. 13, 15
- [31] Tero Karras, Samuli Laine, Miika Aittala, Janne Hellsten, Jaakko Lehtinen, and Timo Aila. Analyzing and improving the image quality of stylegan. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8110–8119, 2020. 2, 3, 4, 5, 7, 14
- [32] Adam R Kosiorek, Heiko Strathmann, Daniel Zoran, Pol Moreno, Rosalia Schneider, Soňa Mokrá, and Danilo J Rezende. Nerf-vae: A geometry aware 3d scene generative model. *arXiv preprint arXiv:2104.00587*, 2021. 3
- [33] Zhengqi Li, Simon Niklaus, Noah Snavely, and Oliver Wang. Neural scene flow fields for space-time view synthesis of dynamic scenes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6498–6508, 2021. 3
- [34] Ji Lin, Richard Zhang, Frieder Ganz, Song Han, and Jun-Yan Zhu. Anycost gans for interactive image synthesis and

- editing. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021. [3](#)
- [35] Gidi Littwin and Lior Wolf. Deep meta functionals for shape representation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 1824–1833, 2019. [3](#)
- [36] Ce Liu, Jenny Yuen, and Antonio Torralba. Sift flow: Dense correspondence across scenes and its applications. *IEEE transactions on pattern analysis and machine intelligence*, 33(5):978–994, 2010. [2](#)
- [37] Michael Mathieu, Camille Couprie, and Yann LeCun. Deep multi-scale video prediction beyond mean square error. *arXiv preprint arXiv:1511.05440*, 2015. [2](#)
- [38] Lars Mescheder, Andreas Geiger, and Sebastian Nowozin. Which training methods for gans do actually converge? In *International conference on machine learning*, pages 3481–3490. PMLR, 2018. [8](#)
- [39] Lars Mescheder, Michael Oechsle, Michael Niemeyer, Sebastian Nowozin, and Andreas Geiger. Occupancy networks: Learning 3d reconstruction in function space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4460–4470, 2019. [3](#)
- [40] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. In *European conference on computer vision*, pages 405–421. Springer, 2020. [2, 3, 4, 14, 15](#)
- [41] Takeru Miyato and Masanori Koyama. cgans with projection discriminator. *arXiv preprint arXiv:1802.05637*, 2018. [5, 8](#)
- [42] Andres Munoz, Mohammadreza Zolfaghari, Max Argus, and Thomas Brox. Temporal shift gan for large scale video generation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 3179–3188, 2021. [3](#)
- [43] Michael Niemeyer, Lars Mescheder, Michael Oechsle, and Andreas Geiger. Differentiable volumetric rendering: Learning implicit 3d representations without 3d supervision. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3504–3515, 2020. [3](#)
- [44] Simon Niklaus, Long Mai, and Feng Liu. Video frame interpolation via adaptive separable convolution. In *IEEE International Conference on Computer Vision*, 2017. [2](#)
- [45] Jeong Joon Park, Peter Florence, Julian Straub, Richard Newcombe, and Steven Lovegrove. DeepSDF: Learning continuous signed distance functions for shape representation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 165–174, 2019. [3](#)
- [46] Keunhong Park, Utkarsh Sinha, Jonathan T Barron, Sofien Bouaziz, Dan B Goldman, Steven M Seitz, and Ricardo Martin-Brualla. Deformable neural radiance fields. *arXiv preprint arXiv:2011.12948*, 2020. [3, 14, 17](#)
- [47] Sunghyun Park, Kangyeol Kim, Junsoo Lee, Jaegul Choo, Joonseok Lee, Sookyoung Kim, and Edward Choi. Vid-ode: Continuous-time video generation with neural ordinary differential equation. *arXiv preprint arXiv:2010.08188*, page online, 2021. [3](#)
- [48] Gaurav Parmar, Richard Zhang, and Jun-Yan Zhu. On buggy resizing libraries and surprising subtleties in fid calculation. *arXiv preprint arXiv:2104.11222*, 2021. [2, 7, 14, 16](#)
- [49] Or Patashnik, Zongze Wu, Eli Shechtman, Daniel Cohen-Or, and Dani Lischinski. Styleclip: Text-driven manipulation of stylegan imagery. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 2085–2094, 2021. [9, 14](#)
- [50] Songyou Peng, Michael Niemeyer, Lars Mescheder, Marc Pollefeys, and Andreas Geiger. Convolutional occupancy networks. In *Computer Vision—ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part III 16*, pages 523–540. Springer, 2020. [3](#)
- [51] Albert Pumarola, Enric Corona, Gerard Pons-Moll, and Francesc Moreno-Noguer. D-nerf: Neural radiance fields for dynamic scenes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10318–10327, 2021. [3](#)
- [52] Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv:1511.06434*, 2015. [7](#)
- [53] Ruslan Rakhimov, Denis Volkhonskiy, Alexey Artemov, Denis Zorin, and Evgeny Burnaev. Latent video transformer. *arXiv preprint arXiv:2006.10704*, 2020. [2](#)
- [54] MarcAurelio Ranzato, Arthur Szlam, Joan Bruna, Michael Mathieu, Ronan Collobert, and Sumit Chopra. Video (language) modeling: a baseline for generative models of natural videos. *arXiv preprint arXiv:1412.6604*, 2014. [2](#)
- [55] Andreas Rössler, Davide Cozzolino, Luisa Verdoliva, Christian Riess, Justus Thies, and Matthias Nießner. Faceforensics: A large-scale video dataset for forgery detection in human faces. *arXiv*, 2018. [2, 3, 6, 13, 17, 20](#)
- [56] Masaki Saito, Eiichi Matsumoto, and Shunta Saito. Temporal generative adversarial nets with singular value clipping. In *Proceedings of the IEEE international conference on computer vision*, pages 2830–2839, 2017. [1, 3, 7, 8, 16](#)
- [57] Masaki Saito, Shunta Saito, Masanori Koyama, and Sotuke Kobayashi. Train sparsely, generate densely: Memory-efficient unsupervised training of high-resolution temporal gan. *International Journal of Computer Vision*, 128:2586–2606, 2020. [1, 2, 3, 6](#)
- [58] Katja Schwarz, Yiyi Liao, Michael Niemeyer, and Andreas Geiger. Graf: Generative radiance fields for 3d-aware image synthesis. *arXiv preprint arXiv:2007.02442*, 2020. [3](#)
- [59] Aliaksandr Siarohin, Stéphane Lathuilière, Sergey Tulyakov, Elisa Ricci, and Nicu Sebe. First order motion model for image animation. *Advances in Neural Information Processing Systems*, 32:7137–7147, 2019. [17](#)
- [60] Vincent Sitzmann, Julien Martel, Alexander Bergman, David Lindell, and Gordon Wetzstein. Implicit neural representations with periodic activation functions. *Advances in Neural Information Processing Systems*, 33, 2020. [2, 3, 4, 5, 14, 15](#)
- [61] Vincent Sitzmann, Michael Zollhöfer, and Gordon Wetzstein. Scene representation networks: Continuous 3d-structure-aware neural scene representations. In *Advances in Neural Information Processing Systems*, 2019. [3](#)

- [62] Ivan Skorokhodov, Savva Ignatyev, and Mohamed Elhoseiny. Adversarial generation of continuous images. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10753–10764, 2021. 3, 4, 13, 14, 17, 19
- [63] Ivan Skorokhodov, Grigorii Sotnikov, and Mohamed Elhoseiny. Aligning latent and image spaces to connect the unconnectable. *arXiv preprint arXiv:2104.06954*, 2021. 3, 13, 14
- [64] Khurram Soomro, Amir Roshan Zamir, and Mubarak Shah. Ucf101: A dataset of 101 human actions classes from videos in the wild. *arXiv preprint arXiv:1212.0402*, 2012. 2, 6, 17, 20
- [65] Nitish Srivastava, Elman Mansimov, and Ruslan Salakhudinov. Unsupervised learning of video representations using lstms. In *International conference on machine learning*, pages 843–852. PMLR, 2015. 2
- [66] Matthew Tancik, Pratul P Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. *arXiv preprint arXiv:2006.10739*, 2020. 2, 3, 4, 5, 14, 15
- [67] Yu Tian, Jian Ren, Menglei Chai, Kyle Olszewski, Xi Peng, Dimitris N. Metaxas, and Sergey Tulyakov. A good image generator is what you need for high-resolution video synthesis. In *International Conference on Learning Representations*, 2021. 1, 2, 3, 5, 7, 16
- [68] Du Tran, Lubomir Bourdev, Rob Fergus, Lorenzo Torresani, and Manohar Paluri. Learning spatiotemporal features with 3d convolutional networks. In *Proceedings of the IEEE international conference on computer vision*, pages 4489–4497, 2015. 16
- [69] Sergey Tulyakov, Ming-Yu Liu, Xiaodong Yang, and Jan Kautz. Mocogan: Decomposing motion and content for video generation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1526–1535, 2018. 1, 2, 3, 4, 5, 7, 9, 16
- [70] Thomas Unterthiner, Sjoerd van Steenkiste, Karol Kurach, Raphael Marinier, Marcin Michalski, and Sylvain Gelly. Towards accurate generative models of video: A new metric & challenges. *arXiv preprint arXiv:1812.01717*, 2018. 2, 7, 16
- [71] Carl Vondrick, Hamed Pirsiavash, and Antonio Torralba. Generating videos with scene dynamics. *Advances in neural information processing systems*, 29:613–621, 2016. 2
- [72] Jacob Walker, Abhinav Gupta, and Martial Hebert. Patch to the future: Unsupervised visual prediction. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, pages 3302–3309, 2014. 2
- [73] Jacob Walker, Ali Razavi, and Aäron van den Oord. Predicting video with vqvae. *arXiv preprint arXiv:2103.01950*, 2021. 2
- [74] Kaisiyuan Wang, Qianyi Wu, Linsen Song, Zhuoqian Yang, Wayne Wu, Chen Qian, Ran He, Yu Qiao, and Chen Change Loy. Mead: A large-scale audio-visual dataset for emotional talking-face generation. In *European Conference on Computer Vision*, pages 700–717. Springer, 2020. 2, 6, 7, 17, 20
- [75] Ting-Chun Wang, Ming-Yu Liu, Jun-Yan Zhu, Guilin Liu, Andrew Tao, Jan Kautz, and Bryan Catanzaro. Video-to-video synthesis. *arXiv preprint arXiv:1808.06601*, 2018. 2
- [76] Yaohui Wang, Piotr Bilinski, Francois Bremond, and Antitza Dantcheva. G3AN: Disentangling appearance and motion for video generation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020. 3
- [77] Yaohui Wang, Francois Bremond, and Antitza Dantcheva. Inmodegan: Interpretable motion decomposition generative adversarial network for video generation. *arXiv preprint arXiv:2101.03049*, 2021. 3
- [78] Dirk Weissenborn, Oscar Täckström, and Jakob Uszkoreit. Scaling autoregressive video models. In *International Conference on Learning Representations*, 2020. 2
- [79] Wenqi Xian, Jia-Bin Huang, Johannes Kopf, and Changil Kim. Space-time neural irradiance fields for free-viewpoint video. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9421–9431, 2021. 3
- [80] Wei Xiong, Wenhan Luo, Lin Ma, Wei Liu, and Jiebo Luo. Learning to generate time-lapse videos using multi-stage dynamic generative adversarial networks. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2018. 2, 6, 13, 17, 20
- [81] Wilson Yan, Yunzhi Zhang, Pieter Abbeel, and Aravind Srinivas. Videogpt: Video generation using vq-vae and transformers. *arXiv preprint arXiv:2104.10157*, 2021. 2, 7, 14, 16
- [82] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *CVPR*, 2018. 14

A. Limitations and potential negative impact

A.1. Limitations

Our model has the following limitations:

- *Limitations of sparse training.* In general, sparse training makes it impossible for D to capture complex dependencies between frames. But surprisingly, it provides state-of-the-art results on modern datasets, which (using the statement from Sec 3.3) implies that they are not that sophisticated in terms of motion.
- *Dataset-induced limitations.* Similar to other machine learning models, our method is bound by the dataset quality it is trained on. For example, for FaceForensics 256² dataset [55], our embedding and manipulations results are inferior to StyleGAN2 ones [1]. This is due to the limited number of identities (just 700) in FaceForensics and their larger diversity in terms of quality compared to FFHQ [30], which StyleGAN2 was trained on.
- *Periodicity artifacts.* G still produces periodic motions sometimes, despite of our acyclic positional embeddings. Future investigation on this phenomena is needed.
- *Poor handling of new content appearing.* We noticed that our generator tries to reuse the content information encoded in the global latent code as much as possible. It is noticeable on datasets where new content appears during a video, like Sky Timelapse or Rainbow Jelly. We believe it can be resolved using ideas similar to ALIS [63].
- *Sensitivity to hyperparameters.* We found our generator to be sensitive to the minimal initial period length $\max_i \sigma_i$ (See Appx B). We increased it for SkyTimelapse [80] from 16 to 256: otherwise it contained unnatural sharp transitions.

We plan to address those limitations in our future works.

A.2. Potential negative impact

The potential negative impact of our method is similar to those of traditional image-based GANs: creating “deep-fakes” and using them for malicious purposes.⁷ Our model made it much easier to train a model which produces much more realistic video samples with a small amount of computational resources. But since the availability of high-quality datasets is very low for video synthesis, the resulted model will fall short compared to its image-based counterpart, which could use rich, extremely qualitative image datasets for training, like FFHQ [30].

⁷<https://en.wikipedia.org/wiki/Deepfake>

B. Implementation and training details

Note, that all the details can be found in the source code: <https://github.com/universome/stylegan-v>.

B.1. Optimization details and hyperparameters

Our model is built on top of the official StyleGAN2-ADA [29] repository⁸. In this work, we build a model to generate continuous videos and a reasonable question to ask was why not use INR-GAN [62] instead (like DI-GAN [5]) to have fully continuous signals? The reason why we chose StyleGAN2 instead of INR-GAN is that StyleGAN2 is amenable to the mixed-precision training, which makes it train ≈ 2 times faster. For INR-GAN, enabling mixed precision severely decreases the quality and we hypothesize the reason if it is that each pixel in INR-GAN’s activations tensor carries more information (due to the spatial independence) since the model cannot spatially distribute information anymore. And explicitly restricting the range of possible values adds a strict upper bound on the amount of information one each pixel is able to carry. We also found that adding coordinates information does not improve video quality for our generator neither qualitatively, nor in terms of scores.

Similar to StyleGAN2, we utilize non-saturating loss and R_1 regularization with the loss coefficient of 0.2 in all the experiments, which is inherited from the original repo and we didn’t try any hyperparameter search for it. We also use the `fmaps` parameter of 0.5 (the original StyleGAN2 used `fmaps` parameter of 1.0), which controls the channel dimensionalities in G and D, since it is the default setting for StyleGAN2-ADA for 256² resolution. This allowed us to further speedup training.

The dimensionalities of w, z, u_t, v_t are all set to 512.

As being stated in the main text, we use a padding-less `conv1d`-based motion mapping network F_m with a large kernel size to generate raw motion codes u_t . In all the experiments, we use the kernel size of 11 and stride of 1. We do not use any dilation in it despite the fact that they could increase the temporal receptive field: we found that varying the kernel size didn’t produce much benefit in terms of video quality. Using padding-less convolutions allows the model to be stable when unrolled at large depths. We use 2 layers of such convolutions with a hidden size of 512. Another benefit of using `conv1d`-based blocks is that in contrast to LSTM/GRU cells one can practically incorporate equalized learning rate [28] scheme into it.

Using `conv1d`-based motion mapping network without paddings forces us to use “previous” motion noise codes z_t^m . That’s why instead of sampling a sequence $z_{t_0}^m, \dots, z_{t_n}^m$, we sample a slightly larger one to adjust for the reduced sequence size. For the same-padding strategy, for sampling

⁸<https://github.com/nvmlabs/stylegan2-ada>

a frame at position $t \in [t_{n-1}, t_n)$, we would need to produce n motion noise codes z^m . But with our kernel size of 11, with 2 layers of convolutions and without padding, the resulted sequence size is $n + 20$.

The training performance of VideoGPT on UCF101 is surprisingly low despite the fact that it was developed for such kind of datasets [81]. We hypothesize that this happens due to UCF101 being a very difficult dataset and VideoGPT being trained with the batch size of 4 (higher batch size didn't fit our 200 GB GPU memory setup), which damaged its ability to learn the distribution.

To train our model, we also utilized adaptive differentiable augmentations of StyleGAN2-ADA [29], but we found it important to make them *video-consistent*, i.e. applying the same augmentation for each frame of a video. Otherwise, the discriminator starts to underperform, and the overall quality decreases. We use the default bgrc augmentations pipe from StyleGAN2-ADA, which includes horizontal flips, 90 degrees rotations, scaling, horizontal/vertical translations, hue/saturation/brightness/contrast changes and luma axis flipping.

While training the model, for real videos we first select a video index and then we select random clip (i.e., a clip with a random offset). This differs from the traditional D-IGAN or VideoGPT training scheme, that's why we needed to change the data loaders to make them learn the same statistics and not get biased by very long videos.

To develop this project, ≈ 7.5 NVidia v100 32GB GPU-years + ≈ 0.3 NVidia A6000 GPU-years were spent.

B.2. Projection and editing procedures

In this subsection, we describe the embedding and editing procedures, which were used to obtain results in Fig 2.

Projection. To project an existing photograph into the latent space of G, we used a procedure from StyleGAN2 [31], but projecting into $\mathcal{W}+$ space [1] instead of $\mathcal{W}$, since it produces better reconstruction results and does not spoil editing properties. We set the initial learning rate to 0.1 and optimized a w code for LPIPS reconstruction loss [82] for 1000 steps using Adam. For motion codes, we initialized a static sequence and kept it fixed during the optimization process. We noticed that when it is also being optimized, the reconstruction becomes almost perfect, but it breaks when another sequence of motion codes is plugged in.

Editing. Our CLIP editing procedure is very similar to the one in StyleCLIP [49], with the exception that we embed an image assuming that it is a video frame in location $t = 0$. On each iteration, we resample motion codes since all our edits are semantic and do not refer to motion. We leave the motion editing with CLIP for future exploration. For the sky editing video presented in Fig 2, we additionally utilize masking: we initialize a mask to cover the trees and try not

to change them during the optimization using LPIPS loss. For all the videos, presented in the supplementary website, *no masking is used*.

The details can be found in the provided source code.

B.3. Additional details on positional embeddings

Mitigating high-frequency artifacts. We noticed that if our periods ω_t are left unbounded, they might grow to very large values (up to magnitude of ≈ 20.0), which corresponds to extra high frequencies (the period length becomes less than 4 frames) and leads to temporal aliasing. That's why we process them via the $\tanh(\omega_t) + 1$ transform: this bounds them into $(0, 2)$ range with the mean of 1.0, i.e. using the at-initialization frequency scaling, which we discuss next.

Linearly spaced periods. An important design decision is the scaling of periods since at initialization it should cover both high-frequency and low-frequency details. Existing works use either exponential scaling $\sigma = (2\pi/2^d, 2\pi/2^{d-1}, \dots)$ (e.g., [22, 40, 46, 63]) or random scaling $\sigma \sim \mathcal{N}(0, \xi I)$ (e.g., [4, 60, 62, 66]). In practice, we scale the i -th column of the amplitudes weight matrix with the value:

$$\sigma_i = \frac{2\pi}{\omega_{\min} + (i/N) \cdot (\omega_{\max} - \omega_{\min})}, \quad (6)$$

where we use $\omega_{\max} = 2^{10}$ frames and $\omega_{\min} = 2^3$ frames in all the experiments, except for SkyTimelapse, for each we use $\omega_{\min} = 2^8$. We call this scheme *linear scaling* and use it as an additional tool to alleviate periodicity since it greatly increases the overall cycle of a positional embedding (see Fig 9). See also the accompanying source code for details.

Another benefit of using our positional embeddings over LSTM is that they are "always stable", i.e. they are always in a suitable range.

C. Evaluation details

For the practical implementation, see the provided source code: <https://github.com/universome/stylegan-v>.

In this section, we describe the difficulties of a fair comparison of the FVD score. There are discrepancies between papers in computing even FID [48]. So, it is less surprising that computing FVD for videos diverge even more and has even more implications for methods evaluation.

First, we note that I3D model [9] has different weights on tf.hub <https://tfhub.dev/deepmind/i3d-kinetics-400/1> — the model which is used in the official FVD repo.⁹ — compared to its official release in the official github repo implementation¹⁰ That's why we manually exported the weights

⁹https://github.com/google-research/google-research/blob/master/frechet_video_distance

¹⁰<https://github.com/deepmind/kinetics-i3d>

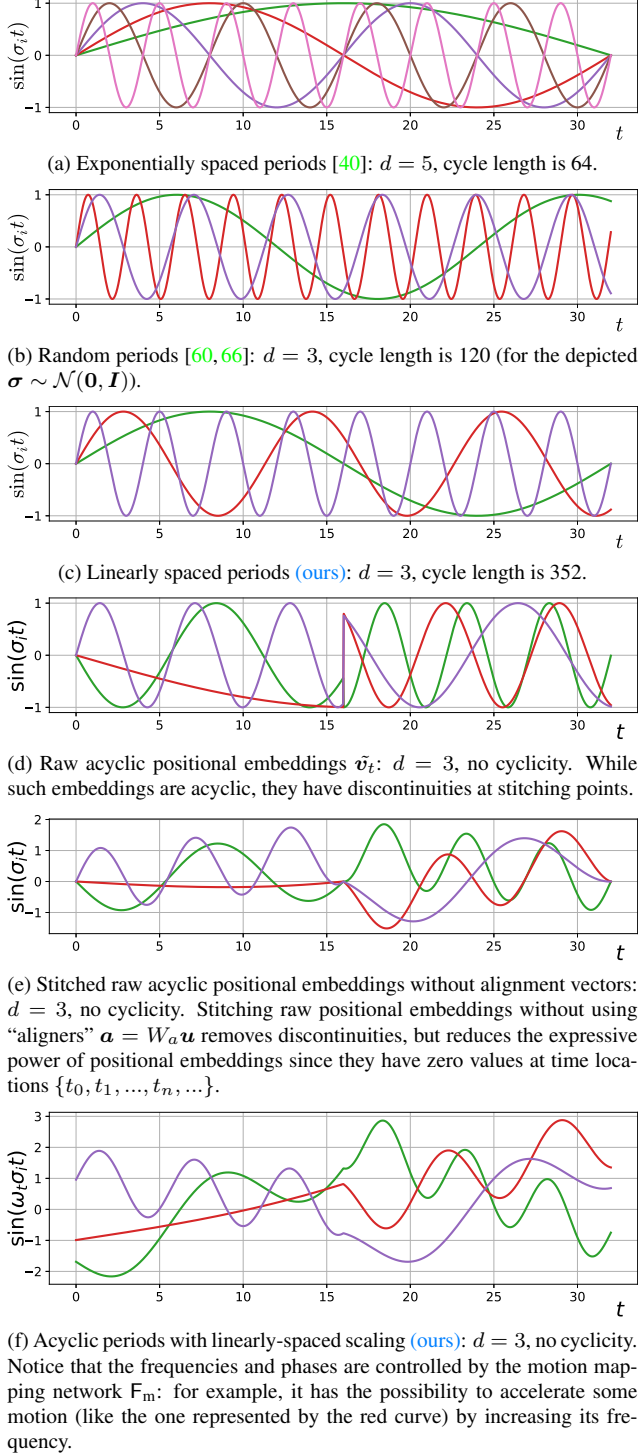


Figure 9. Visualizing positional embeddings $\sin(\sigma t)$ for different initialization strategies of periods scales σ . The cycle length is the minimum value of t for which the positional embedding vector starts repeating itself (it is computed as a least common multiple of all the individual periods lengths). Existing works use either exponentially spaced or random scaling, but in our case we use the linearly spaced one since it has a very large global cycle (in contrast to exponential scaling) and is guaranteed to include high-frequency, medium-frequency and high-frequency waves (in contrast to random scaling).

from tf.hub and used this github repo ¹¹ to obtain an exact implementation in Pytorch.

There are several issues with FVD metric on its own. First, it does not capture motion collapse, which can be observed by comparing FVD_{16} and FVD_{128} scores between StyleGAN-V and StyleGAN-Vwith LSTM motion codes instead of our ones: the latter one has a severe motion collapse issue (see the samples on our website) and has similar or lower FVD_{128} scores compared to our model: 196.1 or 165.8 (depending on the distance between anchors) vs 197.0 for our model. Another issue with FVD calculation is that it is biased towards image quality. If one trains a good image generator, i.e. a model which is not able to generate any videos at all, then FVD will still be good for it even despite the fact that it would have degenerate motion.

We also want to make a note on how we compute FID for video generators. For this, we generate 2048 videos of 16 frames each (starting with $t = 0$) and use all those frames in the FID computation. In this way, it gives $\approx 33k$ images to construct the dataset, but those images will have lower diversity compared to a typically utilized 50k-sized set of images from a traditional image generator [30]. The reason of it is that 16 images in a single clip likely share a lot of content. A better strategy would be to generate 50k videos and pick a random frame from each video, but this is too heavy computationally for models which produce frames autoregressively. And using just the first frame in FID computation will unfairly favour MoCoGAN-HD, which generates the very first frame of each video with a freezed StyleGAN2 model.

FVD is greatly influenced by 1) how many clips per video are selected; 2) with which offsets; and 3) at which frame-rate. For example, SkyTimelapse contains several *extremely* long videos: if we select as many clips as possible from each real video, that it will severely bias the statistics of FVD. For FaceForensics, videos often contain intro frames during their first ≈ 0.5 -1.0 seconds, which will affect FVD when a constant offset of 0 is chosen to extract a single clip per video.

That’s why we use the following protocol to compute FVD_n .

Computing real statistics. To compute real statistics, we select a *single* clip per video, chosen at a random offset. We use the actual frame-rate of the dataset, which the model is being trained on, without skipping any frames. The problem of such an approach is that for datasets with small number of long videos (like, FaceForensics, see Table 7) might have noisy estimates. But our results showed that the standard deviations are always < 3.0 even for FaceForensics 256². The largest standard deviation we observed was when computing FVD_{16} on RainbowJelly: on this dataset it was 26.15 for VideoGPT, but it is $< 1\%$ of its overall

¹¹https://github.com/hassony2/kinetics_i3d_pytorch

Table 4. Subtleties of FVD calculation. We report different ways of calculating FVD₁₆ on FaceForensics 256² (FF) and SktTime-lapse 256² (ST) for one of our checkpoints. We show how the scores of StyleGAN-V are influenced a lot when different strategies of FVD₁₆ calculation are employed. See the text for the description of each row.

Method	FF	ST
Proper computation	76.82 ± 1.57	61.95 ± 0.92
When resized to 128 ²	38.92	59.86
With jpg/png discrepancy	80.17	71.40
When using all clips per video	84.59	72.03
When using only first frames	91.64	59.74
When using subsampling of $s = 8$	82.88	90.21
Still real images	342.5	166.8

magnitude.

Computing fake statistics. To compute fake statistics, we generate 2048 videos and save them as frames in JPEG format via the Pillow library. We use the quality parameter $q = 95$ for doing this, since it was shown to have very close quality to PNG, but without introducing artifacts that would lead to discrepancies [48]. Ideally, one would like to store frames in the PNG format, but in this case it would be too expensive to represent video datasets: for example, MEAD 1024² would occupy ≈ 0.5 terabytes of space in this case.

We illustrate the subtleties of FVD computation in Table 4. For this, we compute real/fake statistics for our model in several different ways:

- *Resized to 128².* Both fake and real statistics images are resized into 128² resolution via the pytorch bilinear interpolation (without corners alignment) before computing FVD.
- *JPG/PNG discrepancy.* Instead of saving fake frames in JPG with $q = 95$, we use $q = 75$ parameter in the PIL library. This creates more JPEG-like artifacts, which, for example, FID is very sensitive to.
- *Using all clips per video.* We use all available n -frames-long clips in each video without overlaps. Note, that our model was trained
- *Using only first frames.* In each real video, instead of using random offsets to select clips, we use the first n frames.
- *Using $s = 8$ subsampling.* When sampling frames for computing real/fake statistics, we select each 8-th frame. This is the strategy which was employed for some of the experiments in the original paper [70] — but in their case, authors trained the model on videos with this subsampling.

Table 5. Inception Score [56] on UCF101 256² (note that the underlying C3D model resizes the 256² videos into 112² resolution under the hood, eliminating high-quality details).

Method	Inception Score [56]
MoCoGAN [69]	10.09 ± 0.30
MoCoGAN+SG2 (ours)	15.26 ± 0.95
VideoGPT [81]	12.61 ± 0.33
MoCoGAN-HD [67]	23.39 ± 1.48
DIGAN [5]	23.16 ± 1.13
StyleGAN-V (ours)	23.94 ± 0.73
Real videos	97.23 ± 0.38

Table 6. FVD₁₆, FID and training costs of modern video generators on FaceForensics 256². Training cost is measured in terms of GPU-days.

Method	FVD ₁₆	FID	Training cost
MoCoGAN [69]	124.7	23.97	5
MoCoGAN+SG2 (ours)	55.62	10.82	8
VideoGPT [81]	185.9	22.7	32
MoCoGAN-HD [67]	111.8	7.12	16.5
DIGAN [5]	62.5	19.1	16
StyleGAN-V (ours)	47.41	9.445	8
StyleGAN2 [29]	N/A	8.42	7.72

For completeness, we also provide the Inception Score [56] on UCF-101 256² dataset in Table 5. Note that is computed by resizing all videos to 112 × 112 spatial resolution (due to the internal structure of the C3D [68] model), which makes it impossible for it to capture high-resolution details of the generated videos, which is the focus of the current work.

In Tab 6, we provide the numbers, used in Fig 3. Note that StyleGAN2 training in our case is slightly slower than the officially specified one (7.3 vs 7.7 GPU days)¹², which we attribute to a slightly slower file system on our computational cluster.

D. Failed experiments

In this section, we provide a list of ideas, which we tried to make work, but they didn’t work either because the idea itself is not good, or because we didn’t put enough experimental effort into investigating it.

Hierarchical motion codes. We tried having several layers of motion codes. Each layer has its own distance between the codes. In this way, high-level codes should capture high-level motion and bottom-level codes should represent short local motion patterns. This didn’t improve the

¹²<https://github.com/NVlabs/stylegan2-ada-pytorch>

scores and didn’t provide any disentanglement of motion information. We believe that the motion should be represented differently (similar to FOMM [59]), rather than with motion codes, because they make it difficult for G to make them temporarily coherent.

Maximizing entropy of motion codes to alleviate motion collapse. As an additional tool to alleviate motion collapse, we tried to maximize entropy of wave parameters of our motion codes. The generator solved the task of maximizing the entropy well, but it didn’t affect the motion collapse: it managed to save some coordination dimensions of v_t specifically to synchronize motions.

Progressive growing of frequencies in positional embeddings. We tried starting with low-frequencies first and progressively open new and new ones during the training. It is a popular strategy for training implicit neural representations on reconstruction tasks (e.g., [22, 46]), but in our case we found the following problem with it. The generator learned to use low frequencies for representing high-frequency motion and didn’t learn to utilize high frequencies for this task when they became available. That’s why high-frequency motion patterns (like blinking or speaking) were unnaturally slow.

Continuous LSTM with EMA states. Our motion codes use sine/cosine activations, which makes them suffer from periodic artifacts (those artifacts are mitigated by our parametrization, but still present sometimes). We tried to use LSTM, but with *exponential moving average* on top of its hidden states to smoothen out motion representations temporally. However, (likely due to the lack of experimental effort which we invested into this direction), the resulted motion representations were either too smooth or too sharp (depending on the EMA window size), which resulted in unnatural motions.

Concatenating spatial coordinates. INR-GAN [62] uses spatial positional embeddings and shows that they provide better geometric prior to the model. We tried to use them as well in our experiments, but they didn’t provide any improvement neither in qualitatively, nor quantitatively, but made the training slightly slower (by $\approx 10\%$) due to the increased channel dimensionalities.

Feature differences in D. Another experiment direction which we tried is computing differences between activations of next/previous frames in a video and concatenating this information back to the activations tensor. The intuition was to provide D information with some sort of “latent” optical flow information. However, it made D too powerful (its loss became smaller than usual) and it started to outpace G too much, which decreased the final scores.

Predicting δ^x instead of conditioning in D. There are two ways to utilize the time information in D: as a conditioning signal or as a learning signal. For the latter one, we tried to predict the time distances between frames by train-

Table 7. Additional datasets information in terms of total lengths (in the total number of hours), average video length (in seconds), frame rate and the amount of speakers (for FaceForensics and MEAD).

Dataset	#hours	avg len	FPS	#speakers
FaceForensics [55]	4.04	20.7s	25	704
SkyTimelapse [80]	12.99	22.1s	25	N/A
UCF-101 [64]	0.51	6.8s	25	N/A
RainbowJelly	7.99	17.1s	30	N/A
MEAD [74]	36.11	4.3s	30	48

ing an additional head to predict the class (we treated the problem as classification instead of regression since there is a very limited amount of time distances between frames which D sees during its training). However, it noticeably decreased the scores.

Conditioning on video length. For *unconditional* UCF-101, it might be very important for G to know the video length in advance. Because some classes might contain very short clips (like, jumping), while others are very long, and it might be useful for G to know in advance which video it will need to generate (since we sample frames at random time locations during training). However, utilizing this conditioning didn’t influence the scores.

E. Datasets details

E.1. Datasets details

We provide the dataset statistics in Fig 10 and their comparison in Table 7. Note, that for MEAD, we use only its front camera shots (originally, it releases shots from several camera positions).

E.2. Rainbow Jelly

For our RainbowJelly benchmark, we used the following film: <https://www.youtube.com/watch?v=P8Bit37hlsQ>. It is an 8-hour-long movie of jellyfish in 4K resolution and 30 FPS from the Hoccori Japan youtube video channel. We cannot release this dataset due to the copyright restrictions, but we released a full script which processes it (see the provided source code). To construct a benchmark, we sliced it into 1686 chunks of 512 frames each, starting with the 150-th frame (to remove the loading screen), center-cropped and resized into 256^2 resolution. This benchmark is advantageous compared to the existing ones in the following way:

1. It contains complex hierarchical motions:
 - a jellyfish flowing in a particular direction (low-frequency global motion);
 - a jellyfish pushing water with its arms (medium-frequency motion)

- small perturbations of jellyfish’s body and tentacles (high-frequency local motion).
2. It is a very high-quality dataset (4K resolution).
 3. It is simple in terms of content, which makes the benchmark more focused on motions.
 4. It contains long videos.

F. Implicit assumptions of sparse training

In this section, we elaborate on our simple theoretical exposition from Sec 3.3

Consider that we want to fit a probabilistic model $q_\theta(\mathbf{x})$ to the real data distribution $\mathbf{x} \sim p(\mathbf{x}) = p(x_1, \dots, x_n)$. For simplicity, we will be considering a discrete finite case, i.e. $n < \infty$, but note that videos, while continuous and infinite in theory, are still discretized and have a time limit to fit on a computer in practice. For fitting the distribution, we use *k-sparse training*, i.e. picking only k random coordinates from each sample $\mathbf{x} \sim p(\mathbf{x})$ during the optimization process. In other words, introducing *k-sparse sampling* reformulates the problem from

$$d(p(\mathbf{x}), q_\theta(\mathbf{x})) \longrightarrow \min_{\theta} \quad (7)$$

into

$$\sum_{I \in \mathcal{I}^k} d(p(\mathbf{x}_I), q_\theta(\mathbf{x}_I)) \longrightarrow \min_{\theta}, \quad (8)$$

where $d(\cdot, \cdot)$ is a problem-specific distance function between probability distributions, $\mathcal{I}^k$ is a collection of all possible sets $I = \{i_1, \dots, i_k\}$ of unique indices $i_j \in \{1, 2, \dots, n\}$ and $\mathbf{x}_I$ denotes a sub-vector $(x_{i_1}, \dots, x_{i_k})$ of $\mathbf{x}$. This means, that instead of bridging together full distributions we choose to bridge all their possible marginals of length k instead. When solving Eq. (8) will help us to obtain the full joint distribution $p(\mathbf{x})$? To investigate this question, we develop the following simple statement.

Let’s denote by $\mathcal{J}_{< i}^k$ a collection of sets J_i of up to k indices s.t. $\forall J_i \in \mathcal{J}_{< i}^k$ we have $j < i$ for all $j \in J_i$.

Using the chain rule, we can represent $p(\mathbf{x})$ as:

$$p(\mathbf{x}) = \prod_{i=1}^n p(x_i | \mathbf{x}_{< i}), \quad (9)$$

where $\mathbf{x}_{< i}$ denotes the sequence $(x_1, \dots, x_{i-1})$. Now, if we know that for each i , there exists $J_i = \{j_1, \dots, j_{k-1}\}$ with $j_\ell < i$ s.t.:

$$p(x_i | \mathbf{x}_{< i}) = p(x_i | \mathbf{x}_{J_i}), \quad (10)$$

then $p(\mathbf{x})$ is obviously simplified to:

$$p(\mathbf{x}) = \prod_{i=1}^n p(x_i | \mathbf{x}_{< i}) = \prod_{i=1}^n p(x_i | \mathbf{x}_{J_i}) \quad (11)$$

Does this tell anything useful? Surprisingly, yes. It says that if $p(\mathbf{x})$ is simple enough that instead of using the whole history $\mathbf{x}_{< i}$ to model $p(x_i | \mathbf{x}_{< i})$ it’s enough to use only some set “representative moments” J_i (unique for each i) with the size $|J_i| < k$, then *k-sparse training* is a viable alternative. After fitting $q_\theta(\mathbf{x})$ via *k-sparse training*, we will be able to obtain $p(\mathbf{x})$ using Eq (10) *even though* $q_\theta(\mathbf{x}) \neq p(\mathbf{x})$! Note, that one can obtain a conditional distributional $p(x_i | \mathbf{x}_I)$ from the marginal one $p(x_i, \mathbf{x}_I)$ for some set of indices $I = \{i_1, \dots, i_{\ell-1}\}$ via:

$$p(x_i | \mathbf{x}_I) = \frac{p(x_i, \mathbf{x}_I)}{p(\mathbf{x}_I)} = \frac{p(x_i, \mathbf{x}_I)}{\int_{\mathbf{x}_I} p(x_i, \mathbf{x}_I) d\mathbf{x}_I}. \quad (12)$$

But we would also like to have the “reverse” dependency, i.e. knowing that if we can approximate the distribution via a set of marginals, then this distribution is not too difficult. For this claim, we will need to consider marginals not of an arbitrary form $p(\mathbf{x}_S)$, but of the form $p(x_i, J_i)$, and we would need exactly n of those. The reverse implication is the following. *If $p(\mathbf{x})$ can be represented as a product of n conditionals $p(i | J_i)$, then for each i there exists $J_i \in \mathcal{J}_{< i}^k$ s.t. $p(x_i | \mathbf{x}_i) = p(x_i | J_i)$.* This statement, just like the previous one, looks obvious. But oddly, requires more than a single sentence to prove. First, we are given that:

$$p(\mathbf{x}) = \prod_{i=1}^n p(x_i | \mathbf{x}_{< i}) = \prod_{i=1}^n p(x_i | \mathbf{x}_{J_i}), \quad (13)$$

but unfortunately, we cannot directly claim that each term in the product $\prod_{i=1}^n p(x_i | \mathbf{x}_{< i})$ equals to its corresponding one in the product $\prod_{i=1}^n p(x_i | \mathbf{x}_{J_i})$. For this, we first need to show that for each m we have:

$$p(\mathbf{x}_{\leq m}) = \prod_{i=1}^m p(x_i | \mathbf{x}_{J_i}) \quad (14)$$

It can be seen from the fact, that:

$$\begin{aligned} p(\mathbf{x}_{\leq m}) &= \int_{\mathbf{x}_{> m}} p(\mathbf{x}) d\mathbf{x}_{> m} \\ &= \int_{\mathbf{x}_{> m}} \prod_{i=1}^n p(x_i | \mathbf{x}_{J_i}) d\mathbf{x}_{> m} \\ &= \prod_{i=1}^m p(x_i | \mathbf{x}_{J_i}) \cdot \int_{\mathbf{x}_{> m}} \prod_{i=m+1}^n p(x_i | \mathbf{x}_{J_i}) d\mathbf{x}_{> m} \\ &= \prod_{i=1}^m p(x_i | \mathbf{x}_{J_i}) \cdot 1 \\ &= \prod_{i=1}^m p(x_i | \mathbf{x}_{J_i}) \end{aligned} \quad (15)$$

This allows to cancel terms in the chain rule one by one, starting from the end, leading to the desired equality:

$$p(x_i | \mathbf{x}_{<i}) = p(x_i | \mathbf{x}_{J_i}) \quad (16)$$

Does this reverse claim tells us anything useful? Surprisingly again, yes. It implies that if we managed to fit $p(\mathbf{x})$ by using k -sparse training, then this distribution is not so sophisticated.

Merging the above two statements together, we see that $p(\mathbf{x})$ can be represented as a product of n conditionals $p(x_i | \mathbf{x}_{J_i})$ for $i = 1, \dots, n$ if and only if for all $i \leq n$ there exists $J_i \in \mathcal{J}_{<i}^{k-1}$ s.t. $p(x_i | \mathbf{x}_{<i}) \equiv p(x_i | \mathbf{x}_{J_i})$.

What does this statement tell for video synthesis? Any video synthesis algorithm utilizes k -sparse training to learn its underlying model, but in contrast to prior work, we use very small values of k . This means, that we fit our model $q_\theta(\mathbf{x})$ to model any k -marginals of $p(\mathbf{x})$ (considering that we pick frames uniformly at random) instead of the full one $p(\mathbf{x})$. And using the above statement, such a setup implies the assumption of Eq (10). This equation says that one can know everything about x_i by just observing previous frames J_i . In other words, x_i must be predictable from $\mathbf{x}_{J_i}$. Moreover, it is easy to show that our statement can generalize to include several $J_i^{(1)}, \dots, J_i^{(\ell)}$ for i -th frame, i.e. there might exist several explainable sets of frames.

G. Additional samples

For the ease of visualization, we provide additional samples of the model via a web page: <https://universome.github.io/stylegan-v>.

H. Comparison to DIGAN

Our model shares a lot of similarities to DIGAN [5] and in this section we highlight those similarities and differences.

H.1. Major similarities

Sparse training. DIGAN also utilizes very sparse training (only 2 frames per video). But in our case, we additionally explore the optimal number of frames per video k (see Sec 3.3).

Continuous-time generator. DIGAN also builds a generator, which is continuous in time. But our generator does not lose the quality at infinitely large lengths.

Dropping conv3d blocks. DIGAN also drops conv3d blocks in their discriminator. But in contrast to us, they still have 2 discriminators.

H.2. Major differences

Motion representation. DIGAN uses only a single global motion code, which makes it *theoretically* impossible to generate infinite videos: at some point it will start

repeating itself (due to the usage of sine/cosine-based positional embeddings). In our case, we use an infinite sequence of motion codes, which are being temporally interpolated, computed wave parameters from and transformed into motion codes. DIGAN mixes temporal and spatial information together into the same positional embedding, which creates the following problem: even when time changes, the spatial location, perceived by the model, *also changes*. This creates a “head-flying-away” effect (see the samples). In our case, we keep these two information sources decomposed from one another.

Generator’s backbone. DIGAN is built on top of INR-GAN [62], while our work uses StyleGAN2. This allows DIGAN to inherit INR-GAN’s benefits from being spatially continuous, but at the expense of being less stable and being slower to train (due to the lack of mixed precision and increased channel dimensionalities from concatenating positional embeddings).

Discriminator structure. DIGAN uses *two* discriminators: the first one operates on image-level and is equivalent to StyleGAN2’s one, while the other one operates on “video” level and takes frames $\mathbf{x}_{t_1}, \mathbf{x}_{t_2}$ and the time differences between them $\Delta = t_2 - t_1$, concatenates them all together into a 7-channel input image (tiling the time difference scalar) and passes into a model with StyleGAN2 discriminator’s backbone. In our case, we utilize a single hypernetwork-based discriminator.

Sampling procedure. We use $k = 3$ samples per video, while DIGAN uses $k = 2$. Also, we sample frames uniformly randomly, while DIGAN selects $t_1 \sim \text{Beta}(2, 1)$ and $t_2 \sim \text{Beta}(1, 2)$ (in this way, DIGAN sometimes have $t_1 > t_2$). Apart from that, they use $T = 16$.

Apart from those major distinctions, there are lot of small implementation differences. We refer an interested reader to the released codebases for them:

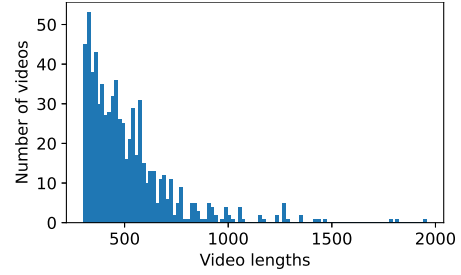
- StyleGAN-V: <https://github.com/universome/stylegan-v>
- DIGAN: <https://openreview.net/forum?id=Czsdv-S4-w9>

H.3. A note on the computational cost

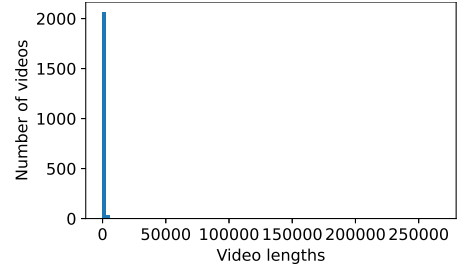
INR-GAN demonstrated that it has higher throughput than StyleGAN2 in terms of images/second [62]. But the authors compare to the original StyleGAN2 implementation and not to the one from StyleGAN2-ADA repo, which is *much* better optimized. Also, they use caching of positional embeddings which is only possible at test-time and has great influence on its computational performance. In this way, we found that that StyleGAN2 is ≈ 2 times faster to train and is *less* consuming in terms of GPU memory than INR-GAN.

DIGAN is based on top of INR-GAN and that’s why suffers from the issues described above. We trained it for

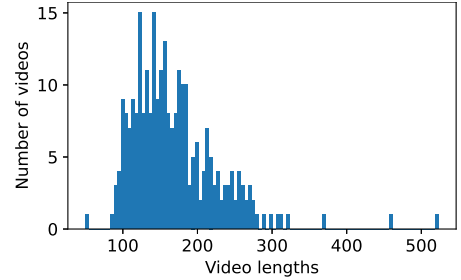
a week on $\times 4$ v100 NVidia GPUs and observed that it stopped improving after ≈ 5 days of training. This is equivalent to $\approx 20k$ real frames seen by the discriminator (while MoCoGAN+SG2 and StyleGAN-V reach $\approx 25k$ in just 2 days for the same resolution in the same environment). For the time of the submitting the main paper, there was no information about the training cost. However, the authors updated their manuscript for the time of submitting the supplementary and specify the training cost of 8 GPU-days 128^2 resolution, which is consistent with our experiments (considering that we have twice as larger resolution).



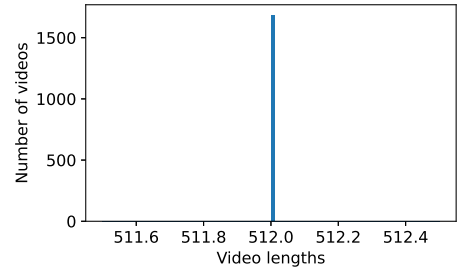
(a) FaceForensics [55].



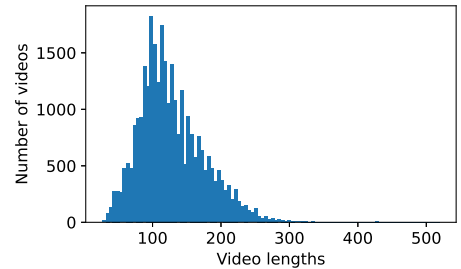
(b) SkyTimelapse [80].



(c) UCF-101 [64].



(d) RainbowJelly [64].



(e) MEAD [64].

Figure 10. Distribution of video lengths (in terms of numbers of frames) for different datasets. Note that RainbowJelly and MEAD [74] are 30 FPS, while the rest are 25 FPS datasets. Note that SkyTimelapse contains several very long videos which might bias the distribution if not treated properly.